

Relational Database Management System

Table of Contents

Part 1: SQL – Structured Query Language.....	3
Session 1: Basic Select.....	3
1: Select only	3
2: Select one table	3
3: Select distinct.....	4
4: Select order by	4
5: Select where	5
6: SELECT RANGE	6
7: SELECT NULL.....	6
8: SELECT LIKE.....	7
Session 2: Sub Query	8
1: Single value	8
2: Multiple values	8
3: ALL ANY	8
Session 3: Aggregate Functions.....	9
1: Count.....	9
2: Min, Max, Sum, Avg.....	10
Session 4: JOIN	11

1: Cartesian	11
2: Inner	12
3: Outer	13
Part 2: Database Design	14
Session 1: Design Principles	14
1: Overview	14
2: Anomalies	16
3: Mantras of Design	16
4: Structure View	17
Session 2: Relationships	17
1: One Table	17
2: Two Tables – 1-N Relationship	18
Part 3: Programming	20
Session 1: Variables	20
Session 2: View	21
Session 3: Function and Procedure	21
Session 4: Trigger	22

Part 1: SQL – Structured Query Language

Session 1: Basic Select

1: Select only

- DB Engine cung cấp câu lệnh SELECT dùng để:
 - + In ra màn hình một cái gì đó, tương đương System.out.println(...) bên Java.
 - + In ra dữ liệu có trong table (hàng/cột).
 - + Dùng cho mục đích nào thì kết quả hiển thị luôn là một table.

SELECT N'Ngô Huỳnh Tấn Lộc is ' + CONVERT(nvarchar, (YEAR(GETDATE()) – 2003)) + ' years old.' AS [My profile]

2: Select one table

- Một Database là nơi chứa data (bán hàng, thư viện, quản lý sinh viên).
- Data được lưu dưới dạng table, tách thành nhiều table (nghệ thuật design DB, NF).
- Dùng lệnh SELECT để xem/in dữ liệu từ table, cũng hiển thị dưới dạng table.
- Cú pháp chuẩn:
 - **SELECT * FROM <TÊN-TABLE>**
* đại diện cho việc tôi muốn lấy all of columns.
- Cú pháp mở rộng:
 - **SELECT TÊN-CÁC-CỘT-MUỐN-LẤY, CÁCH-NHAU-BỞI-DẤU-PHẪY FROM <TÊN-TABLE>**
 - **SELECT CÓ THỂ DÙNG CÁC HÀM XỬ LÝ CÁC CỘT ĐỂ ĐỘ LẠI THÔNG TIN HIỂN THỊ FROM <TÊN-TABLE>**
- Data trả về cell/ô có NULL, hiểu rằng là chưa xác định value/giá trị, chưa biết, chưa có, từ từ cập nhật sau. Ví dụ: Sinh viên kí tên vào danh

sách thi, cột điểm ngay lúc vừa kí tên gọi là NULL, mang trạng thái chưa xác định.

```
SELECT EmployeeID, LastName + ' ' + FirstName AS [Full Name],  
Title, BirthDate FROM Employees
```

3: Select distinct

- Khi ta SELECT ít cột từ table gốc, có nguy cơ dữ liệu sẽ bị trùng lại, không phải do ta sai, không phải do người thiết kế table sai và người nhập liệu sai, mà là do chúng ta có nhiều thông tin trùng nhau/đặc điểm trùng nhau, nên nếu chỉ tập trung vào data này thì trùng nhau chắc chắn xảy ra.
- Vd: 100 triệu người dân Việt Nam được quản lí các thông tin bao gồm: ID, Name, DOB, ... Tỉnh thành sinh sống. 100 triệu người nhưng sinh sống tại 34 tỉnh thành.
- Lệnh SELECT hỗ trợ loại trừ dòng trùng nhau/trùng 100%:
SELECT DISTINCT TÊN-CÁC-CỘT FROM <TÊN-TABLE>
SELECT DISTINCT Country FROM Customers

4: Select order by

- Ta muốn sắp xếp dữ liệu/sort theo tiêu chí nào đó, thường là tăng dần ASCENDING – ASC, hoặc giảm dần DESCENDING – DESC, mặc định không nói gì cả thì là sort ASC.
**SELECT TÊN-CÁC-CỘT FROM <TÊN-TABLE> ORDER BY
TÊN-CỘT-MUỐN-SORT <KIỂU-SORT>**
SELECT * FROM Orders ORDER BY Freight DESC

5: Select where

- Mệnh đề WHERE dùng để làm bộ lọc/filter/nhặt ra những dữ liệu ta cần theo tiêu chí nào đó. Vd: lọc ra những bạn sinh viên có quê ở TP HCM, lọc ra những bạn sinh viên có quê ở Hà Nội và gpa ≥ 8 .
- Cú pháp bộ lọc:
SELECT TÊN-CÁC-CỘT-BẠN-MUỐN-IN-RA FROM <TÊN-TABLE> WHERE <ĐIỀU KIỆN LỌC>
 - + Điều kiện lọc: tìm từng dòng với cái cột có giá trị cần lọc, lọc theo tên cột với value thế nào, lấy tên cột, xem value trong cell có thỏa điều kiện lọc hay không.
- Để viết điều kiện lọc ta cần:
 - + Tên cột.
 - + Value của cột (cell).
 - + Toán tử (operator) $>$, $\geq$, $<$, $\leq$, $=$, $\neq$, $<>$ ($\neq$ hoặc $<>$ đều là so sánh khác nhau), nếu nhiều điều kiện lọc đi kèm, ta dùng thêm logic operators: AND, OR, NOT.
- Lọc liên quan đến giá trị/value/cell chứa gì, ta cần quan tâm đến data types:
 - + Số: nguyên/thực: ghi rõ ra như truyền thống. Vd: 1, 2, 3.14, -10, ...
 - + Chuỗi/kí tự: 'A', 'Ahihi', ...
 - + Ngày tháng: 'YYYY-MM-DD', '2003-01-01', ...
- Công thức full của SELECT:
SELECT ... FROM ... WHERE ... GROUP BY ... HAVING... ORDER BY ...
 - + SELECT có thể dùng thêm DISTINCT, các hàm xử lí các cột, nested query, sub query.
 - + From từ một hoặc N table.

- Rất cẩn thận khi trong mệnh đề WHERE có AND OR trộn với nhau, ta phải xài thêm () để phân tách thứ tự filter:

SELECT ... FROM ... (SO SÁNH AND OR KHÁC) AND (SO SÁNH AND OR KHÁC)

SELECT * FROM Customers WHERE Country = 'USA' OR Country = 'Italy'

6: SELECT RANGE

- Khi ta cần lọc dữ liệu trong một đoạn cho trước, thay vì dùng $\geq$... AND $\leq$..., ta có thể thay thế bằng mệnh đề BETWEEN, IN.

- Cú pháp BETWEEN:

... WHERE TÊN-CỘT BETWEEN VALUE-1 AND VALUE-2

+ BETWEEN thay thế cho 2 mệnh đề $\geq$ AND $\leq$.

SELECT * FROM Orders WHERE Freight BETWEEN 100 AND 200

- Cú pháp IN:

... WHERE TÊN-CỘT IN (Một tập các giá trị được liệt kê cách nhau bởi dấu phẩy)

+ IN thay thế cho một loạt OR.

+ Chỉ khi ta liệt kê được tập các giá trị thì mới chơi IN, còn khoảng số thực thì không làm được.

SELECT * FROM Customers WHERE Country IN ('UK', 'USA')

7: SELECT NULL

- Trong thực tế có những lúc dữ liệu/thông tin chưa xác định được nó là gì. Vd: Sinh viên kí tên vào danh sách thi, cột điểm ngay vừa lúc kí tên gọi là NULL, mang trạng thái chưa xác định.

- Hiện tượng dữ liệu chưa xác định, chưa biết, từ từ đưa vào sau, hiện nhìn vào chưa thấy có data, thì ta gọi giá trị chưa xác định này là NULL.
- NULL đại diện cho thứ chưa xác định, chưa xác định tức là không có giá trị, không có giá trị thì không thể so sánh >, >=, <, <=, =, !=, <>.
- Cấm tuyệt đối dùng các toán tử so sánh kèm với giá trị NULL.
- Ta dùng toán tử IS NULL, IS NOT NULL, NOT (... IS NULL) để filter cell có giá trị NULL.

SELECT * FROM Employees WHERE NOT (Region IS NULL)

8: SELECT LIKE

- Trong thực tế có những lúc ta muốn tìm dữ liệu/filter theo kiểu gần đúng, gần đúng trên chuỗi, ví dụ: liệt kê ai đó có tên là AN, khác với câu liệt kê ai đó có tên bắt đầu bằng chữ A.
- Tìm đúng: toán tử = 'AN', tìm gần đúng, tìm có sự xuất hiện, không dùng =, ta dùng toán tử LIKE.
- Để sử dụng toán tử LIKE, ta cần thêm 2 thứ hỗ trợ, dấu % và dấu _.
 + % đại diện cho 1 chuỗi bất kì nào đó.
 + _ đại diện cho 1 kí tự bất kì nào đó.
 + Vd:
 - ... Name LIKE 'A%', bất kì ai có tên bắt đầu bằng chữ A, phần còn lại không quan tâm.
 - ... Name LIKE 'A_', bất kì ai có tên gồm 2 kí tự, trong đó kí tự đầu tiên phải là A.

**SELECT * FROM Employees WHERE Name LIKE '_____' OR
Name LIKE '%_____'**

-- Liệt kê các nhân viên mà từ cuối cùng trong tên gồm 5 kí tự.

Session 2: Sub Query

1: Single value

- Cú pháp: **SELECT * FROM <TÊN-TABLE> WHERE ...**
 - + WHERE Cột = value nào đó.
 - + WHERE Cột LIKE pattern nào đó, vd: Name LIKE 'A%'.
 - + WHERE Cột BETWEEN range.
 - + WHERE Cột IN (một tập giá trị được liệt kê).
- Một câu SELECT tùy cách viết có thể trả về đúng 1 giá trị/cell. Một câu SELECT tùy cách viết có thể trả về đúng 1 tập giá trị/cell, tập kết quả này đồng nhất (các giá trị khác nhau của 1 biến).
- WHERE Cột = value nào đó, ta có thể thay value này bằng một câu SQL khác miễn trả về 1 cell, kỹ thuật viết câu SQL theo kiểu hỏi gián tiếp, lồng nhau, trong câu SQL này chứa câu SQL khác.

SELECT * FROM Orders WHERE EmployeeID = (SELECT EmployeeID FROM Employees WHERE FirstName = 'Robert')

2: Multiple values

- Cú pháp:
SELECT * FROM <TÊN-TABLE> WHERE CỘT IN (MỘT CÂU SELECT TRẢ VỀ 1 CỘT NHIỀU DÒNG – NHIỀU VALUE CÙNG KIỂU – TẬP HỢP)
SELECT * FROM Orders WHERE EmployeeID IN (SELECT EmployeeID FROM Employees WHERE City = 'London')

3: ALL ANY

- Cú pháp:

SELECT * FROM <TÊN-TABLE> WHERE CỘT > >= < <= ALL (1 CÂU SUB 1 CỘT NHIỀU VALUE)

SELECT * FROM Orders WHERE Freight >= ALL (SELECT Freight FROM Orders)

SELECT * FROM <TÊN-TABLE> WHERE CỘT > >= < <= ANY (1 CÂU SUB 1 CỘT NHIỀU VALUE)

Session 3: Aggregate Functions

1: Count

- DB Engine hỗ trợ một loạt nhóm hàm dùng để thao tác trên nhóm dòng/cột, gom data, tính toán trên đám data được gom này, nhóm hàm gom nhóm, Aggregate Functions, Aggregation: COUNT(), SUM(), MIN(), MAX(), AVG().
- Cú pháp chuẩn:
SELECT TÊN-CÁC-CỘT, HÀM GOM NHÓM() ... FROM <TÊN-TABLE>
- Cú pháp mở rộng:
SELECT TÊN-CÁC-CỘT, HÀM GOM NHÓM() ... FROM <TÊN-TABLE> WHERE ... GROUP BY (GOM THEO CỤM CỘT NÀO) HAVING ...
- Hàm COUNT(): dùng để đếm số lần xuất hiện của một cái gì đó.
 - + COUNT(*) FROM ... : đếm số dòng trong table, đếm tất cả các dòng, không quan tâm tiêu chuẩn nào khác.
 - + COUNT(*) FROM ... WHERE ... : chọn những dòng thỏa tiêu chí WHERE nào đó trước đã rồi mới đếm, filter trước rồi mới đếm.

+ COUNT(CỘT NÀO ĐÓ): nếu cell của cột chứa null thì không tính, không đếm.

- Sub query mới: xem một câu SELECT thành một table, biến table này vào trong mệnh đề FROM, và nhớ đặt tên giả cho table này.

SELECT COUNT(*) FROM (SELECT DISTINCT City FROM Employees) AS CITIES

- Khi câu hỏi có tính toán gom data (hàm aggregate) mà lại chứa từ khóa “MỖI”, gặp từ “MỖI” chính là chia để đếm, chia để trị, chia cụm để gom đếm. Tức là đếm theo sự xuất hiện của nhóm, count++ ở nhóm này thôi, sau đó reset lại ở nhóm mới.

SELECT City, COUNT(City) AS [Number of employees] FROM Employees GROUP BY City

- Khi xài hàm gom nhóm, bạn có quyền liệt kê tên cột lẻ ở SELECT, nhưng cột lẻ đó phải bắt buộc xuất hiện trong mệnh đề GROUP BY để đảm bảo logic: CỘT HIỆN THỊ | SỐ LƯỢNG ĐI KÈM, đếm, gom theo cột hiện thị mới hợp lý. Nếu bạn gom theo Key/Primary Key là vô nghĩa, vì key không trùng nhau, mỗi thằng mỗi nhóm, đếm cái gì.
- Filter sau khi đếm. WHERE sau đếm, WHERE sau khi đã gom nhóm, aggregate thì gọi là HAVING.

SELECT City, COUNT(*) AS [No emps] FROM Employees GROUP BY City HAVING COUNT(*) >= 2

2: Min, Max, Sum, Avg

- Min(): tìm giá trị nhỏ nhất trong 1 cột.

SELECT * FROM Employees WHERE BirthDate = (SELECT MIN(BirthDate) FROM Employees)

- Max(): tìm giá trị lớn nhất trong 1 cột.

**SELECT * FROM Orders WHERE Freight = (SELECT
MAX(Freight) FROM Orders)**

- Sum(): tính tổng giá trị của 1 cột.

SELECT SUM(Freight) AS [Freight in total] FROM Orders

- Avg(): tính giá trị trung bình của 1 cột.

**SELECT * FROM Orders WHERE Freight >= (SELECT
AVG(Freight) FROM Orders)**

Session 4: JOIN

1: Cartesian

CREATE DATABASE Cartesian

CREATE TABLE VnDict

**(
 Nmbr int,
 VnDesc nvarchar(30)
)**

INSERT INTO VnDict VALUES(1, N'Một')

INSERT INTO VnDict VALUES(2, N'Hai')

CREATE TABLE EnDict

**(
 Nmbr int,
 EnDesc nvarchar(30)**

)

INSERT INTO EnDict(1, 'One')

INSERT INTO EnDict(2, 'Two')

DROP TABLE EnDict

➤ **DDL: data definition language**

- JOIN là gộp chung thành một table tạm trong ram, không ảnh hưởng dữ liệu gốc của mỗi table, JOIN là SELECT cùng lúc nhiều table. Nên đặt tên giả ALIAS (AS) để tham chiếu, chỉ định cột thuộc table tránh nhầm lẫn.

- Cú pháp viết thứ 1:

SELECT v.*, e.* FROM VnDict v ,EnDict e

- Cú pháp viết thứ 2:

SELECT v.*, e.* FROM VnDict v CROSS JOIN EnDict e

➤ **DML: data manipulation language.**

➤ **Sinh table mới, tạm thời lúc chạy thôi, số cột bằng tổng hai bên, số dòng bằng tích hai bên.**

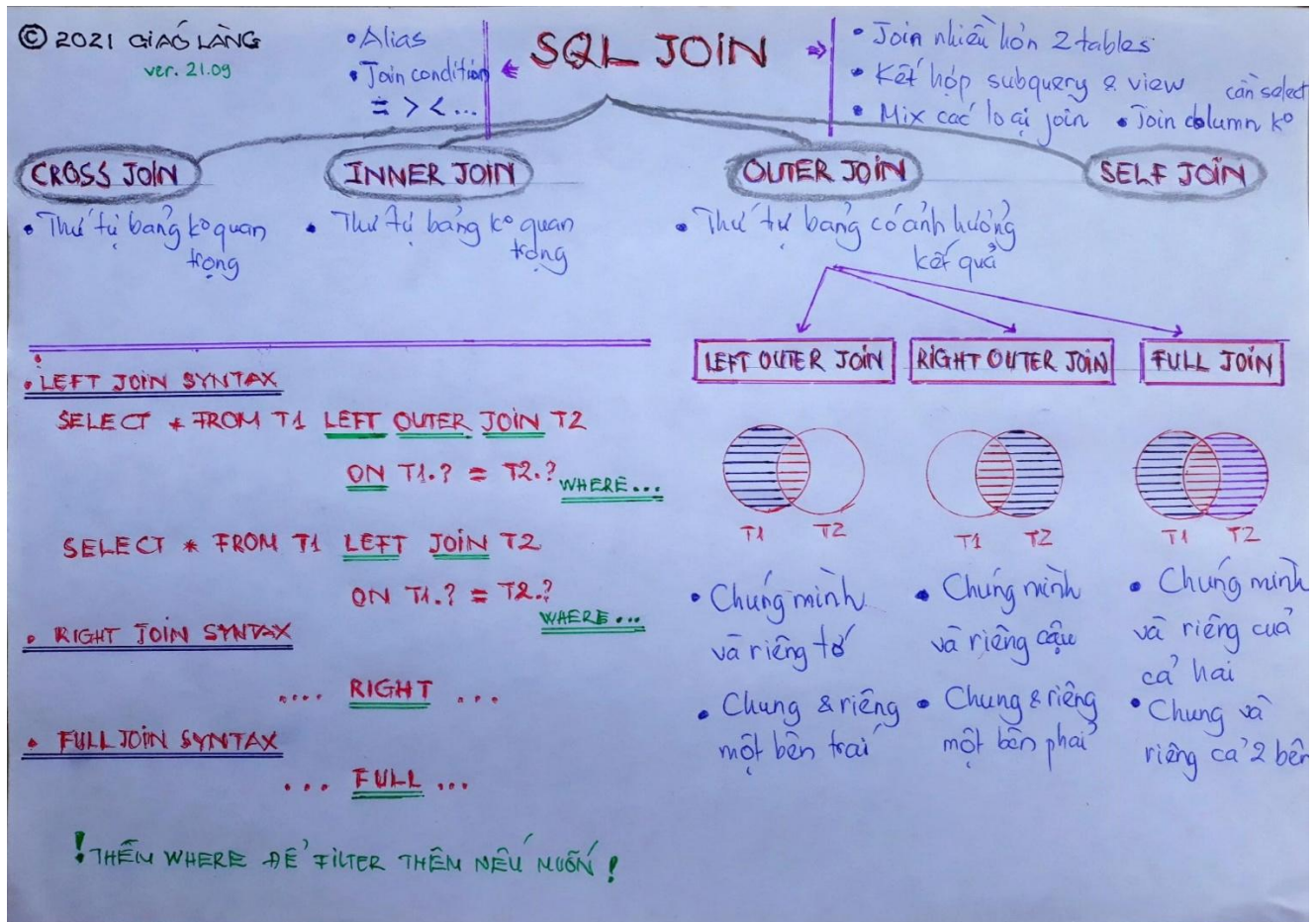
2: Inner

- Inner Join là ghép có chọn lọc, ghép “môn đăng hộ đối”, ghép trên điểm chung, cột chung, dùng toán tử =.

SELECT * FROM VnDict v INNER JOIN EnDict e ON v.Nmbr = e.Nmbr

SELECT * FROM VnDict v JOIN EnDict e ON v.Nmbr = e.Nmbr

3: Outer



- Outer Join sinh ra để đảm bảo việc kết nối ghép bảng không bị mất mát data, do Inner Join, Join = chỉ tìm cái chung hai bên.

- Left Outer Join: chung và riêng một bên trái.

```
SELECT * FROM EnDict e LEFT JOIN VnDict v ON e.Nmbr = v.Nmbr
```

```
SELECT * FROM EnDict e LEFT OUTER JOIN VnDict v ON e.Nmbr = v.Nmbr
```

- Right Outer Join: chung và riêng một bên phải.

```
SELECT * FROM EnDict e RIGHT JOIN VnDict v ON e.Nmbr = v.Nmbr
```

SELECT * FROM EnDict e RIGHT OUTER JOIN VnDict v ON e.Nmbr = v.Nmbr

- Full Outer Join: chung và riêng cả 2 bên.

SELECT * FROM EnDict e FULL JOIN VnDict v ON e.Nmbr = v.Nmbr

SELECT * FROM EnDict e FULL OUTER JOIN VnDict v ON e.Nmbr = v.Nmbr

Part 2: Database Design

Session 1: Design Principles

1: Overview

I. Requirements Gathering/Problem Defined

Object – Programming World

Student **an**

id: SE12345
 firstName: An
 lastName: Nguyễn
 yob: 2003

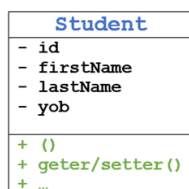
Entity – Database World

Student **binh**

id: SE12346
 firstName: Binh
 lastName: Lê
 yob: 2003

II. Conceptual Design in OOP

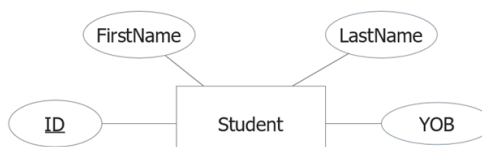
Class Diagram



Class

II. Conceptual Design in Database

ERD – Entity Relationship Diagram (Chen or Crow's Foot notations)



Entity

III. Logical Design & Implementation

```

public class Student {
    private String id;
    private String firstName;
    private String lastName;
    private int yob;
    //...
}
  
```

Class

III. Logical Design & Implementation

```

CREATE TABLE Student
(
    ID char(8) PRIMARY KEY,
    FirstName nvarchar(10) NOT NULL,
    LastName nvarchar(30) NOT NULL,
    YOB int CHECK (YOB BETWEEN 1950 AND 2003)
)
  
```

Table

III. Relational Model

$$R = \{attr1, attr2, attr3, \dots\}$$

$$Student (ID, FirstName, LastName, YOB)$$

Relation Schema

IV. Execution

```

public static void main(String[] args) {
    Student an = new Student("SE123456", "An", "Nguyễn", 2003);
    Student binh = new Student("SE123457", "Binh", "Lê", 2003);

    System.out.printf("%-8s%-10s%-30s%-4s\n",
        "ID", "First Name", "Last Name", "YOB");
    an.showProfile();
    binh.showProfile();
}
  
```

Java Language

IV. Execution/Populating

```

INSERT INTO Student
VALUES ('SE123456', 'N'AN', 'N'NGUYỄN', 2003)
  
```

```

INSERT INTO Student
VALUES ('SE123457', 'N'BINH', 'N'LÊ', 2003)
  
```

```

-- Selection
SELECT * FROM STUDENT
  
```

```

-- Projection
SELECT ID, FirstName, LastName FROM Student
  
```

SQL – Structured Query Language

IV. Execution behind the Scenes

Selection: SELECT có WHERE

$$S := \sigma_C(R)$$

C là điều kiện filter (WHERE)

Projection: SELECT lấy cột

$$S := \pi_{A1, A2, \dots, An}(R)$$

A1, A2, ... là cột muốn lấy

Cartesian product and Joins

$$R3 := R1 \times R2$$

$$R3 := R1 \bowtie_{\text{join condition}} R2$$

Relational Algebra

```

F001:
|ID|First Name|Last Name|YOB|
|SE123456|An|Nguyễn|2003|
|SE123457|Binh|Lê|2003|
  
```

	ID	FirstName	LastName	YOB
1	SE123456	AN	NGUYỄN	2003
2	SE123457	BINH	LÊ	2003

2: Anomalies

© 2021 GIÁO LÃNG
ver. 21.10

Tính dị thường của dữ liệu - Anomalies

1. Redudancy (tính dư thừa, trùng lặp):

* Cụm thông tin chuyên ngành bị lặp lại ở mỗi sinh viên

2. Insertion Anomaly (tính dị thường khi thêm dữ liệu):

* Chuyên ngành Ngôn ngữ Hàn mới thành lập, chưa tuyển sinh làm sao đưa vào table?

* Thêm sinh viên ngành SE, nhưng thông tin ngành học nhập không nhất quán

3. Deletion Anomaly (tính dị thường khi xóa dữ liệu):

* Sinh viên ngành Ngôn ngữ Nhật rút hồ sơ/ngỉ học, vậy chuyên ngành Ngôn ngữ Nhật còn tồn tại?

4. Update Anomaly (tính dị thường khi cập nhật dữ liệu):

* Thay đổi Hotline của mỗi bộ môn, phải nhớ đồng bộ trên tất cả các hồ sơ sinh viên của bộ môn

Lí giải nguyên nhân (hãy xem nguyên lí thiết kế để khắc chế):

* Do gộp nhiều thứ "không thuộc về nhau" vào chung một chỗ.

* Nấu "lẩu" nếm gia vị không khéo thành "chối" nhau

	StudentID	Last Name	First Name	DOB	Address	MajorID	Vn-MajorName	En-MajorName	Website	Hotline
1	7777	777	777	NULL	NULL	KR	Ngôn ngữ Hàn	Korean	https://kr-en.homuni.fpt.edu.vn	094x
2	GD07	Đinh	Bảy	NULL	NULL	GD	Thiết kế đồ họa	Graphic Design	https://gd-en.homuni.fpt.edu.vn	090x
3	GD08	Huyên	Tam	NULL	NULL	GD	Thiết kế đồ họa	Graphic Design	https://gd-en.homuni.fpt.edu.vn	090x
4	JP09	Ngô	Chín	NULL	NULL	JP	Ngôn ngữ Nhật	Japanese	https://jp-en.homuni.fpt.edu.vn	093x
5	SE01	Nguyễn	Mai	NULL	NULL	SE	Kỹ thuật phần mềm	Software Engineering	https://se-en.homuni.fpt.edu.vn	090x
6	SE02	Lê	Hai	NULL	NULL	SE	Kỹ thuật phần mềm	Software Engineering	https://se-en.homuni.fpt.edu.vn	090x
7	SE03	Tiến	Ba	NULL	NULL	SE	Kỹ thuật phần mềm	Software Engineering	https://se-en.homuni.fpt.edu.vn	090x
8	SE04	Phạm	Bốn	NULL	NULL	IA	An toàn thông tin	Information Assurance	https://ia-en.homuni.fpt.edu.vn	091x
9	SE05	Lý	Năm	NULL	NULL	IA	An toàn thông tin	Information Assurance	https://ia-en.homuni.fpt.edu.vn	091x
10	SE06	Võ	Sáu	NULL	NULL	IA	An toàn thông tin	Information Assurance	https://ia-en.homuni.fpt.edu.vn	091x
11	SE10	Đào	Mười	NULL	NULL	SE	Kỹ thuật phần mềm	Software Engineering	https://se-en.homuni.fpt.edu.vn	6789

3: Mantras of Design

© 2021 GIÁO LÃNG
ver. 21.10

Khẩu quyết thiết kế CSDL/Table Mantras of Design

1. Object & Entity Recognition

* Nhận diện đối tượng/thực thể qua các đặc điểm được mô tả, biến và value

2. Single Responsibility - SR (S in SOLID)

* Người nào việc nấy, không ai ôm đồm

3. Decomposition

* Phân rã, tách bảng để đạt SR

4. Vertical Expansion over Horizontal Expansion

* Mở rộng theo chiều dọc bảng (thêm dòng) tốt hơn thêm cột

5. Inheritance

* Kế thừa và chiến lược mapping lên xuống

6. Normalization Form

* Dạng chuẩn 3NF đã là đủ

7. Một dòng không làm nên table

8. Log table & History

* Nhìn lịch sử, chiều thời gian tồn tại của dữ liệu mà thiết kế ra bảng

9. Status, Catalog, Type, Enumeration

* Những table nho nhỏ

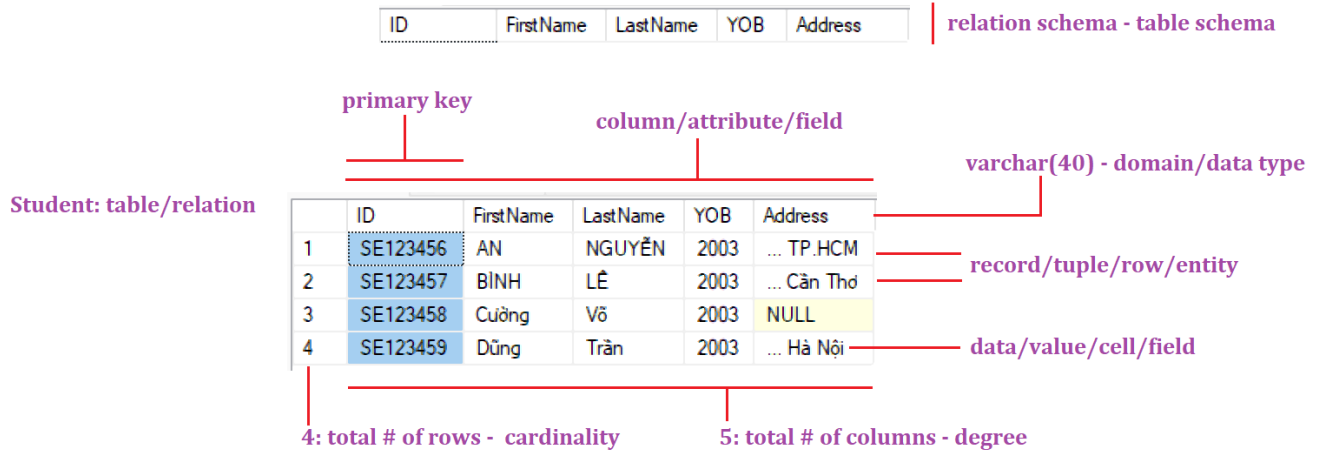
10. Relationship

* Một này mấy kia, đọc từ hai phía

4: Structure View

© 2021 GIÁO LÃNG
ver. 21.10

Góc nhìn cấu trúc dữ liệu Structure View



Session 2: Relationships

1: One Table

- CREATE dùng để tạo cấu trúc lưu trữ/giàn khung/thùng chứa dùng để lưu trữ data/info, tương đương việc xây một phòng chứa đồ - database, mua tủ bỏ vào phòng - table. Một database chứa nhiều table - 1 ngôi nhà có nhiều phòng - 1 phòng có nhiều tủ.

CREATE DATABASE DBDESIGN_ONETABLE

- Tạo cấu trúc lưu trữ - chưa nói data bỏ vào - DDL - Data Definition Language (phần nhánh của SQL). Thao tác trên data/món đồ trong tủ/table - DML (Data Manipulation Language)/DQL (Data Query Language) - dành riêng cho SELECT.
- Constraints là cách ta/DB Designer ép cell/cột nào đó phải có value như thế nào. Đặt ra quy tắc/rule cho việc nhập data.

- Vì có nhiều quy tắc, nên để tránh nhầm lẫn, dễ kiểm soát, ta có quyền đặt tên ràng buộc, constraint name. Mặc định DB Engine tự đặt tên cho các ràng buộc mà nó nhìn thấy khi ta gõ lệnh DDL, nó cũng cho mình cơ chế tự đặt tên ràng buộc.
- Thêm ràng buộc Primary Key:
 - + Cách 1: **StudentID char(8) PRIMARY KEY**
 - + Cách 2:
**StudentID char(8),
PRIMARY KEY (StudentID)**
 - + Cách 3:
**StudentID char(8),
CONSTRAINT PK_STUDENT PRIMARY KEY (StudentID)**
 - + Cách 4:
**CREATE TABLE Student(...)
ALTER TABLE Student ADD CONSTRAINT PK_STUDENT
PRIMARY KEY (StudentID)**
 - + Xóa ràng buộc khóa chính:
ALTER TABLE Student DROP CONSTRAINT PK_STUDENT

2: Two Tables – 1-N Relationship

- Tạo table 1 trước, table N tạo sau, còn xóa thì xóa table N trước, rồi xóa table 1 sau.
- Tên cột khóa ngoại/tham chiếu không cần trùng bên PK của table 1, nhưng bắt buộc phải trùng 100% kiểu dữ liệu.
- Thêm ràng buộc Foreign Key:
 - + Cách 1:
MajorID char(8) REFERENCES Major(MajorID)
 - + Cách 2:

MajorID char(8) FOREIGN KEY REFERENCES Major(MajorID)

+ Cách 3:

MajorID char(8),

CONSTRAINT FK_STUDENT_MAJOR FOREIGN KEY (MajorID) REFERENCES Major(MajorID)

+ Cách 4:

CREATE TABLE Student (...)

ALTER TABLE Student ADD CONSTRAINT

FK_STUDENT_MAJOR FOREIGN KEY (MajorID)

REFERENCES Major(MajorID)

+ Xóa ràng buộc khóa ngoại:

ALTER TABLE Student DROP CONSTRAINT

FK_STUDENT_MAJOR

- Có 3 cách giải thêm hành xử khi xóa, sửa data ở ràng buộc khóa ngoại và dính đến khóa chính: CASCADE, SET NULL, SET DEFAULT

+ CASCADE: hiệu ứng DOMINO, sụp đổ dây chuyền, Table 1 xóa, Table N đi sạch, Table 1 sửa, Table N bị sửa theo.

+ SET NULL: Khi Table 1 xóa, Table N về NULL.

+ SET DEFAULT: Khi Table 1 xóa, Table N về DEFAULT.

➤ Nên ON DELETE SET NULL và ON UPDATE CASCADE

```
CREATE TABLE StudentV3
(
    StudentID char(8) PRIMARY KEY,
    LastName nvarchar(40) NOT NULL,
    FirstName nvarchar(10) NOT NULL,
    DOB datetime NULL,
    Address nvarchar(50) NULL,
    MajorID char(2) DEFAULT 'SE',
    CONSTRAINT FK_StudentV3_MajorV3 FOREIGN KEY (MajorID) REFERENCES MajorV3(MajorID) ON DELETE SET DEFAULT ON UPDATE CASCADE
)
-- CHO SV KHÔNG CHỌN CHUYÊN NGÀNH, HẸN HỌC NGÀNH GÌ? HỌC SE ĐẤY
INSERT INTO StudentV3(StudentID, LastName, FirstName) VALUES ('SE2', 'PHAM', 'BINH')

ALTER TABLE StudentV3 ADD CONSTRAINT FK_StudentV3_MajorV3 FOREIGN KEY (MajorID) REFERENCES MajorV3(MajorID) ON DELETE CASCADE ON UPDATE CASCADE
```

- Xóa và sửa data:

```
-- UPDATE DML, MẠNH MẼ, SỬA DATA ĐANG CÓ.
UPDATE MajorV3 SET MajorID = 'AK' -- CẦN THẬN NẾU KHÔNG CÓ WHERE, TOÀN BỘ TABLE BỊ ẢNH HƯỞNG
-- TRỪ UPDATE CỘT KEY
UPDATE MajorV3 SET MajorID = 'AK' WHERE MajorID = 'AH'|

-- SỤP ĐỔ, XÓA 1, N ĐI SẠCH SẼ.
-- XÓA CHUYÊN NGÀNH AHIHI, XEM SAO, CÒN SV NÀO KHÔNG.
DELETE FROM MajorV3 WHERE MajorID = 'AK' -- SV AH1 LÊN ĐƯỜNG LUÔN
```

Part 3: Programming

Session 1: Variables

```
-- KHAI BÁO BIẾN
GO

DECLARE @msg1 AS nvarchar(30)

DECLARE @msg nvarchar(30) = N'Xin chào - Welcome to T-SQL'

-- IN BIẾN CÓ 2 LỆNH
PRINT @msg -- IN RA KẾT QUẢ BÊN CỬA SỐ CONSOLE GIỐNG LẬP TRÌNH

SELECT @msg -- IN RA KẾT QUẢ DƯỚI DẠNG TABLE

DECLARE @yob int -- = 2003

-- GÁN GIÁ TRỊ CHO BIẾN
SET @yob = 2003

SELECT @yob = 2004 -- SELECT DÙNG 2 CÁCH: GÁN GIÁ TRỊ CHO BIẾN, IN GIÁ TRỊ CỦA BIẾN

PRINT @yob

IF @yob > 2003
    BEGIN -- {
        PRINT 'Hey, Boy/Girl'
        PRINT 'Hello Gen Z'
    END -- }
ELSE
    PRINT 'Hello Lady & Gentlemen'

GO
```

Session 2: View

- Khi có nhiều câu lệnh SQL phức tạp, hay ta cần viết lại nhiều lần một câu SELECT nào đó, ta đặt tên cho câu lệnh SQL SELECT này, sau này muốn xài lại câu SQL SELECT này chỉ gọi tên ra là được.

```
GO
```

```
CREATE VIEW VW_LondonEmployees AS SELECT * FROM Employees WHERE City = 'London'
```

```
GO
```

```
-- XÀI VIEW, COI MÀY LÀ TABLE, VÌ SAU LÚNG MÀY LÀ 1 CÂU SELECT CHỐNG LÚNG.
```

```
SELECT * FROM VW_LondonEmployees
```

Session 3: Function and Procedure

- RDBMS có hai loại hàm:
 - + Stored Procedure tương đương hàm void.
 - + Function: hàm có trả về một giá trị qua lệnh RETURN.

```
GO
```

```
CREATE PROC PR_InsertEvent
```

```
@name nvarchar(30)
```

```
AS
```

```
    INSERT INTO [Event] VALUES (@name)
```

```
GO
```

```
-- CHÈN CÁC DÒNG VÀO TABLE
```

```
EXEC PR_InsertEvent @name = N'Bí quyết dùng source ở FE'
```

```
EXEC PR_InsertEvent @name = N'Hồ Sen chờ ai'
```

GO

```

CREATE FUNCTION FN_C2F(@cDegree float)
RETURNS float
AS
BEGIN
    DECLARE @fDegree float = @cDegree * 1.8 + 32
    RETURN @fDegree
END

```

GO

```

-- Sử dụng hàm, hàm là phải viết kèm với lệnh khác.
PRINT dbo.FN_C2F(37)
PRINT N'37 độ C là ' + CONVERT(nvarchar, dbo.FN_C2F(37)) + N' độ F'
PRINT N'37 độ C là ' + CAST(dbo.FN_C2F(37) AS nvarchar) + N' độ F'

```

Session 4: Trigger

- Trigger là một hàm void, không nhận tham số, không trả về, nó làm nhiệm vụ gác cổng một table nào đó. Nếu có sự thay đổi data trong table, nó sẽ tự động thực thi/chạy, dùng để kiểm tra hay đảm bảo tính toàn vẹn/nhất quán/consistency của dữ liệu hoặc dùng để kiểm tra các ràng buộc mà SQL chuẩn không thể cung cấp đủ.
- Chỉ tự gọi liên quan đến table nào đó và liên quan đến ba lệnh INSERT, UPDATE, DELETE. Gắn chặt với một table, nhưng không cấm code của nó can thiệp table. Một table không nhất thiết phải có Trigger.
- DB Engine sẽ tự tạo ra hai table “ảo” lúc runtime liên quan đến Trigger:
 - + Câu lệnh INSERT vào table => DB Engine tạo ra một table ảo tên là INSERTED, chứa record vừa đưa vào từ câu lệnh INSERT.
 - + Câu lệnh DELETE FROM TABLE => DB Engine tạo ra một table ảo tên là DELETED, chứa những dòng vừa bị xóa.

- + Câu lệnh UPDATE [TÊN-TABLE] SET CỘT = VALUE => DB Engine tạo hai table ảo: INSERTED chứa value mới đưa vào, DELETED chứa value cũ/bị ghi đè.
- Liên quan đến table, có hai loại Trigger cơ bản:
 - + Chặn trước khi dữ liệu đưa vào table, lúc này dữ liệu mới vào INSERTED (BEFORE).
 - + Chặn sau khi đã vào INSERTED và đồng thời vào luôn table rồi (AFTER).

GO

```

CREATE TRIGGER TR_CheckInsertionLimitationOnEvent ON Event
FOR INSERT
AS
BEGIN
    -- Kiểm tra xem trong table Event không cho vượt quá 5 sự kiện.
    -- If số sự kiện > 5 thì rollback!!!
    -- Phải đếm số sự kiện đang có!
    -- Lấy được số sự kiện ra để if, tức là khai báo biến,
    -- nhớ lệnh count(*) trong SELECT TRẢ VỀ 1 TABLE
    DECLARE @noEvents int
    SELECT @noEvents = COUNT(*) FROM [Event]
    -- PRINT @noEvents
    -- SELECT @noEvents
    IF @noEvents > 5
    BEGIN
        PRINT 'To much events. No more 5 events!!!'
        ROLLBACK
    END
END

GO

```